

W6M1

1. Define Pipeline Success and Failure First

- How did your team define success and failure?
- Did any part of the pipeline log an error but still finish successfully?

1. Define Pipeline Success and Failure First

A pipeline is not successful simply because the DAG finishes without an error. The team must first define what data and results are required for a successful run.

- Define success as collecting, cleaning, aggregating, and loading the data for the two target dates into PostgreSQL and passing the required data quality checks.
- Define failure conditions such as API errors, missing data, invalid required fields, aggregation errors, or database load failures.
- Decide whether invalid records should be excluded or whether they should fail the entire run.

2. Divide the DAG into Tasks That Can Run and Retry Independently

- Why did your team divide the tasks this way?
- If one task fails, where does the pipeline resume?
- How is data passed between tasks?

2. Divide the DAG into Tasks That Can Run and Retry Independently

A task is not simply a section of code. It is a unit that Airflow executes, monitors, and retries independently.

- Decide how to divide extraction, cleaning, aggregation, loading, and validation into tasks.
- Clearly define the input, output, and dependencies of each task.
- Make sure retrying one task does not damage previous results or create duplicates.
- Use XCom for small metadata such as file paths or record counts, not for the dataset itself.

3. Distinguish the DAG Run Time from the Data Processing Period

- Which dates does each DAG run process?
- Where is the processing period defined?
- Can the actual target dates be confirmed in the logs?

3. Distinguish the DAG Run Time from the Data Processing Period

The time when Airflow runs the DAG may be different from the dates of the data being processed.

- Understand the meaning of `logical_date`, `data_interval_start`, and `data_interval_end`.
- Define whether the start and end boundaries are inclusive or exclusive.
- Check that UTC and KST conversion does not shift the target dates.
- Retries and reruns must use the same processing period.

4. Verify Data Quality at Every Stage

- Which data quality rules did your team apply?
- Were invalid records excluded, corrected, or treated as a failure?
- How did you verify that the final aggregates were correct?

4. Verify Data Quality at Every Stage

Do not assume that the result is correct simply because the final table exists. Validate the data during extraction, cleaning, aggregation, and loading.

- Check that the required data for both target dates was collected.
- Validate required columns, null values, type conversions, and invalid values during cleaning.
- Confirm that the result contains one row per station and that relevant totals are preserved before and after aggregation.
- After loading, check for empty results, duplicate keys, and missing target dates.

5. Keep Results Safe After Failures and Reruns

- Which data quality rules did your team apply?
- Were invalid records excluded, corrected, or treated as a failure?
- How did you verify that the final aggregates were correct?

5. Keep Results Safe After Failures and Reruns

Do not assume that the result is correct simply because the final table exists. Validate the data during extraction, cleaning, aggregation, and loading.

- Check that the required data for both target dates was collected.
- Validate required columns, null values, type conversions, and invalid values during cleaning.
- Confirm that the result contains one row per station and that relevant totals are preserved before and after aggregation.
- After loading, check for empty results, duplicate keys, and missing target dates.

6. Document Team Decisions with Reasons and Evidence

- Which decisions changed after your team inspected the actual data?
- Why did your team add or exclude particular features?
- What evidence proves that the pipeline satisfies the mission requirements?

6. Document Team Decisions with Reasons and Evidence

What matters is not the number of technologies or features used, but how the team interpreted the requirements and justified its decisions.

- Separate mandatory requirements from features added by the team.
- Record the selected approach, alternatives considered, reasons, and limitations in the team wiki.
- Document deferred features and the conditions under which they should be reconsidered.
- Provide evidence such as Airflow runs, logs, quality-check results, SQL queries, and rerun results.